POLY PMNA · DIPLOMA CURRICULUM REVISION 2026

Course 1131 | Semester I | Practicum | 4.5 Credits

Problem Solving and Python Programming

A complete student handbook for structured problem solving, flowcharts, Python fundamentals, control structures, data structures and modular programming.

Programmes	**AI / CS / IT / Robotics family**
Teaching scheme	**3:0:3:0**
Contact hours	**90**
Theory + Practical	**45 + 45**
CIE / ESE	**40 / 100**
Total assessment	**140**

AUTHORITY AND VERIFICATION

Official Source Declaration and Verified Inconsistencies

This handbook is based only on the supplied SITTTR Kerala **Diploma Curriculum Revision 2026** PDF for Course **1131**, **Problem Solving and Python Programming**. No Revision 2021 lesson content has been merged.

Official-source truncation: The supplied four-page PDF ends midway through the detailed syllabus for Module III, after the list practical. The summary table on page 2 confirms all four modules and their hours, but the detailed subtopic table does not include the remaining Module III dictionary/set rows or any Module IV detailed rows. This handbook covers those missing details strictly from the official “Syllabus - Major Topics” table, not from another revision.

Terminology inconsistency: Page 1 labels the teaching scheme as *L:T:P:R*, while page 2 defines the fourth column as *S (Project)*. Both show 3:0:3:0, so navigation displays the numeric scheme and records the label mismatch here.

Assessment presentation: Page 1 shows CIE 40 and ESE 100. Page 2 splits these into Theory 20+60=80 and Practical 20+40=60, giving total 140. These figures are complementary, not contradictory.

Not specified in the supplied official syllabus PDF: prescribed textbooks, reference books, websites, equipment list beyond ordinary computer-lab resources, and an official model question-paper pattern.

Course at a Glance

90 h
Total semester contact

45 h
Theory

45 h
Practical

4.5
Credits

Official teaching and assessment scheme

L	T	P	S/R	Self-learning/week	CIE Theory	ESE Theory	CIE Practical	ESE Practical	Total
3	0	3	0	6	20	60	20	40	140

How to use this handbook

1. Read the concept and inspect the flowchart or code.

2. Use Malayalam support to confirm meaning, then run the code yourself.

3. Attempt the self-check and practical task before reading the answer.

OFFICIAL INTENT

Objectives and Course Outcomes

Course objectives

1. Design structured algorithms and visual flowcharts before implementation.
2. Understand Python syntax, variables, data types, operators, selection and iteration.
3. Manipulate strings, lists, sets, tuples and dictionaries efficiently.

4. Use built-in and user-defined functions for modular, reusable programs and scope management.
5. Develop practical programming skill through laboratory and real-life problems.

Course outcomes

CO1: Design algorithms/flowcharts and develop programs with variables, basic data types and standard I/O.

CO2: Use control structures and loop-control statements strategically.

CO3: Manage strings, lists, tuples, sets and dictionaries using operations and built-in methods.

CO4: Develop modular programs using functions, modules, packages, scope and argument passing.

Official Bloom's level for all COs: **Apply**.

TRACEABILITY

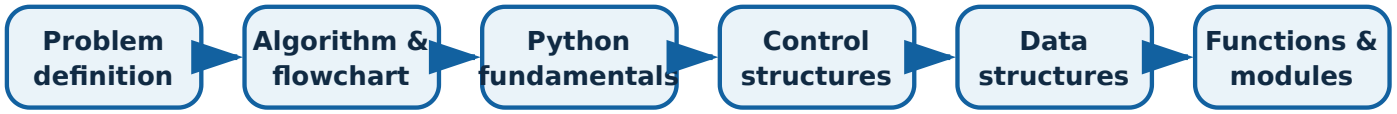
Curriculum Coverage Map

Module	Official topics	Hours L+P	CO	Handbook section
I	Problem solving, development cycle, algorithms, flowcharts, Python introduction, installation, variables, types, operators, I/O	12+12	CO1	Module I
II	if, if-else, nested if, if-elif, match-case, for, while, range, nested loops, break/continue/pass	12+12	CO2	Module II
III	Strings, lists, tuples, sets, dictionaries, operations, slicing and methods	12+12	CO3	Module III
IV	Built-in/user-defined functions, arguments, scope, modules, packages, NumPy introduction	9+9	CO4	Module IV

Module I Module II Module III Module IV Practical Centre Questions Quick Revision

LEARNING ROADMAP

From Problem to Reusable Program



Each module adds one engineering habit: define, design, implement, control, organise and reuse.

MODULE I

12 L | 12 P | CO1

Problem Solving & Python Fundamentals

A reliable program starts before coding. First define the problem, identify inputs and outputs, design an algorithm, validate it with a flowchart, and only then write Python.

Problem-solving methods and program development cycle–

Algorithmic method follows a finite, ordered and reproducible sequence. A **heuristic** is a practical rule-of-thumb that may find a good answer quickly but does not guarantee the best answer.

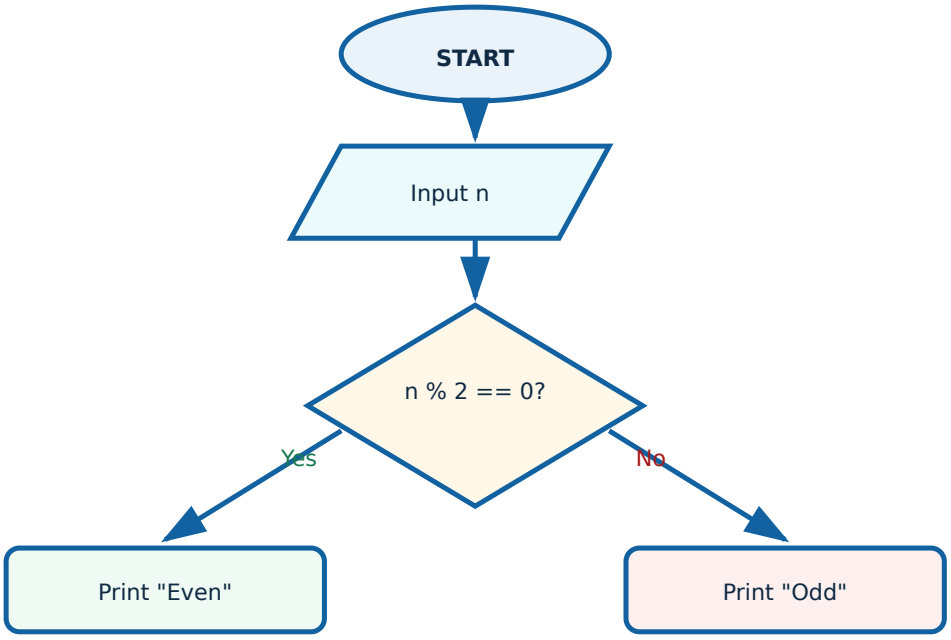
The development cycle is: problem analysis → design → coding → testing/debugging → documentation → maintenance.

Malayalam support: Coding തുടങ്ങുന്നതിന് മുമ്പ് input, output, constraints എന്നിവ വ്യക്തമാക്കുക. Algorithmic method definite steps ആണ്; heuristic ഒരു practical shortcut ആണ്.

Common mistake: Starting code without defining test cases. Write at least normal, boundary and invalid-input cases first.

Algorithms and flowcharts–

A good algorithm is finite, unambiguous, effective and produces defined output. Standard flowchart symbols: oval for Start/End, parallelogram for input/output, rectangle for processing, diamond for decision and arrows for flow.



Decision flowchart for checking parity.

Python installation, interpreter and IDEs–

Python source files use `.py`. The interpreter executes statements and reports syntax/runtime errors. IDLE is bundled with Python; VS Code and PyCharm provide richer editing, debugging and project tools.

1. Install an institution-approved Python 3 release.
2. Verify with `python --version` or `py --version`.
3. Create `hello.py`, save it, then run it.

```
print("Python is ready")
```

> **Malayalam support:** IDE code എഴുതാനുള്ള environment ആണ്; Python Interpreter ആണ് code execute ചെയ്യുന്നത്. ഇവ രണ്ടും ഒരേ കാര്യമല്ല.

Variables, literals, data types and conversion–

Variables are names bound to objects. Common built-in types are `int`, `float`, `bool`, `str` and NoneType. Python is dynamically typed, but a variable can be rebound to a different type.

```
age = 18
temperature = 36.5
is_ready = True
name = "Anu"
value = None
print(type(age))
count = int("25")
```

`input()` always returns a string. Convert before arithmetic.

Operators, input/output and f-strings–

Operator groups: arithmetic, assignment, comparison, logical, identity, membership and bitwise. Use == for value equality and `is` mainly for identity checks such as `x is None`.

```
principal = float(input("Principal: "))
rate = float(input("Rate (%): "))
time = float(input("Time (years): "))
interest = principal * rate * time / 100
print(f"Simple interest = {interest:.2f}")
```

Application: f-strings make engineering output readable and allow controlled decimal formatting.

Worked demonstration: Celsius to Fahrenheit

Formula: F = (9/5)C + 32

```
c = float(input("Celsius: "))
f = (9 / 5) * c + 32
print(f"{c:.1f} °C = {f:.1f} °F")
```

For 25 °C, result = 77.0 °F.

Self-check

Why does `"5" + "2"` produce `"52"`?

Both operands are strings, so + performs concatenation. Use `int("5") + int("2")` for 7.

MODULE II

12 L | 12 P | CO2

Control Structures & Looping Structure

Selection: if, if-else, nested if and if-elif–

Selection chooses a path based on a Boolean condition. Use `if` for one-way decisions, `if-else` for two-way decisions and `if-elif-else` for mutually exclusive alternatives.

```
mark = float(input("Mark: "))
if not 0 <= mark <= 100:
    print("Invalid mark")
elif mark >= 90:
    print("A")
elif mark >= 75:
    print("B")
elif mark >= 60:
    print("C")
else:
    print("Needs improvement")
```

Malayalam support: `elif` chain-ൽ ആദ്യത്തെ True condition മാത്രം execute ചെയ്യും. Conditions specific-to-general order-ൽ എഴുതുക.

Do not use separate `if` blocks when only one grade should print.

match-case–

`match-case` provides pattern-based multi-way selection in modern Python.

```
day = int(input("Day number (1-7): "))
match day:
    case 1: print("Monday")
    case 2: print("Tuesday")
    case 3: print("Wednesday")
    case 4: print("Thursday")
    case 5: print("Friday")
    case 6: print("Saturday")
    case 7: print("Sunday")
    case _: print("Invalid")
```

for loop, range() and nested loops–

`for` iterates over a sequence. `range(start, stop, step)` excludes `stop`. Nested loops are useful for tables and patterns.

```
n = int(input("Number: "))
for i in range(1, 11):
    print(f"{n} × {i} = {n*i}")

for row in range(1, 5):
    print("*" * row)
```

> `range(5, 0, -1)` gives 5, 4, 3, 2, 1.

while loop and loop-control statements–

`while` repeats while a condition remains true. `break` exits the nearest loop, `continue` skips to the next iteration, and `pass` is a do-nothing placeholder.

```
total = 0
while True:
    text = input("Enter number or q: ")
    if text.lower() == "q":
        break
    if not text.lstrip("-").isdigit():
        print("Skipped invalid input")
        continue
    total += int(text)
print("Total:", total)
```

> Every `while` loop must have a condition-changing path, otherwise it can become infinite.

Course [18]

Field validation for simple interest

```
p = float(input("Principal: "))
r = float(input("Rate: "))
t = float(input("Time: "))
if p <= 0 or r < 0 or t < 0:
    print("Invalid field")
else:
    print("SI =", p*r*t/100)
```

First Series Test preparation

Practise one program each on variables/I/O, if-elif, for loop, while loop and input validation. Predict output before running.

MODULE III

Strings & Data Structures - List, Tuple, Set and Dictionary

12 L **12 P** **CO3**

Strings: indexing, slicing and methods–

Strings are immutable sequences of Unicode characters. Indexing selects one character; slicing selects a range using `s[start:stop:step]`.

```
text = "Polytechnic"
print(text[0])
print(text[-1])
print(text[0:4])
print(text[::-1])
print(text.upper())
print(text.lower())
print(text.split("t"))
print(text.strip())
print(text.count("y"))
```

> **Malayalam support:** String immutable ആണ്. Method ഉപയോഗിച്ചാൽ original string മാറില്ല; പുതിയ string return ചെയ്യും.

Lists and list methods–

Lists are mutable ordered collections. They support indexing, slicing, concatenation, repetition and membership.

```
marks = [72, 85, 61]
marks.append(90)
marks.insert(1, 78)
marks.remove(61)
last = marks.pop()
marks.sort()
print(marks, last)
```

`list.sort()` changes the list and returns `None`. Use `sorted(list)` when you need a new list.

Tuples–

Tuples are ordered but immutable. Use them for fixed records, coordinates and values that should not be changed accidentally.

```
point = (10, 20)
x, y = point
single = (5,)
```

Sets and set operations–

Sets store unique, unordered elements. They are ideal for removing duplicates and performing union, intersection and difference.

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a.union(b))
print(a.intersection(b))
print(a.difference(b))
```

Malayalam support: Set-ൽ duplicate values സൂക്ഷിക്കില്ല. Index position ആശ്രയിച്ച code എഴുതരുത്.

Dictionaries and dictionary methods–

A dictionary maps unique keys to values. It is suitable for student records, component parameters and configuration data.

```
student = {"name": "Anu", "mark": 84}
student["grade"] = "B"
student["mark"] = 88
print(student.keys())
print(student.values())
print(student.items())
removed = student.pop("grade")
print(student)
```

Real-life use: Store sensor readings as `{"temperature": 31.2, "humidity": 68}`.

Structure	Ordered	Mutable	Duplicates	Best use
str	Yes	No	Yes	Text
list	Yes	Yes	Yes	Changing sequence
tuple	Yes	No	Yes	Fixed record
set	No positional order	Yes	No	Uniqueness and set algebra
dict	Insertion order	Yes	Keys unique	Key-value records

MODULE IV

Functional and Modular Programming in Python

9 L | 9 P | CO4

Built-in and user-defined functions–

Functions divide a program into named, reusable units. Official built-ins include `len()`, `min()`, `max()` and `sum()`.

```
def average(values):
    if not values:
        return None
    return sum(values) / len(values)

data = [72, 84, 91]
print(average(data))
```

> A function should normally return a value rather than print internally when reuse is required.

Positional and default arguments–

```
def simple_interest(p, r=8.0, t=1.0):
    return p * r * t / 100

print(simple_interest(10000))
print(simple_interest(10000, 9.5, 2))
```

Required positional arguments must come before default arguments in the function definition.

Local and global scope–

A local variable exists inside its function. A global variable is defined at module level. Prefer parameters and return values over modifying globals.

```
tax_rate = 0.18

def total_with_tax(amount):
    total = amount * (1 + tax_rate)
    return total
```

> **Malayalam support:** Function-ന്റെ ഉള്ളിലെ local variable പുറത്തുനിന്ന് നേരിട്ട് ഉപയോഗിക്കാൻ കഴിയില്ല. Data parameter ആയി നൽകുകയും result return ചെയ്യുകയും ചെയ്യുക.

Modules, packages and importing—

A module is a Python file. A package organises related modules. Import only what is needed and avoid naming your file the same as a standard module.

```
# geometry.py
def rectangle_area(length, width):
    return length * width

# main.py
from geometry import rectangle_area
print(rectangle_area(5, 3))
```

Introduction to NumPy—

NumPy is a third-party package for efficient numerical arrays and vectorised operations. Install only through the institution-approved environment.

```
import numpy as np

values = np.array([10, 20, 30])
print(values * 2)
print(values.mean())
```

> NumPy is not part of the Python standard library. A missing-package error requires installation in the same interpreter environment used by the IDE.

RULE AND PROCEDURE BANK

Python Quick Rules

Input rule

`input()` returns `str`. Validate and convert.

Range rule

Stop value is excluded: `range(1,5)` gives 1-4.

Equality rule

== compares values; `is` compares identity.
Course 1131

Mutation rule

List/dict/set can change; string/tuple cannot.

Function rule

Single purpose, clear parameters, useful return value.

Debug rule

Read the last traceback line first, reproduce with a minimal test.

DIAGRAM LIBRARY

One-Look Visuals

Selection vs iteration

Selection: choose one path once. **Iteration:** repeat a path until sequence ends or condition changes.

Data-structure decision

Text → string; changing ordered data → list; fixed ordered record → tuple; unique values → set; labelled fields → dictionary.

PRACTICAL CENTRE

Laboratory Experiments and Record Format

Use the institution's laboratory rules. Exact software version and machine configuration must be supplied by the instructor because they are not specified in the PDF.

Lab 1: Laboratory safety and Python setup

Aim: Follow lab rules, install/configure Python and an IDE, verify execution.

1. Inspect workstation and cables; do not alter mains connections.
2. Log in using authorised credentials.
3. Verify Python version.
4. Create and run a one-line program.
5. Record version and output.

Result: Python environment verified.

Lab 2: Algorithms and flowcharts

Tasks: Simple interest and odd/even check.

Record: Problem, inputs, outputs, algorithm, flowchart, test data, expected result.

Viva: Why must a flowchart decision have labelled exits? To make each outcome and path unambiguous.

Lab 3: Basic Python programs

Arithmetic operations, simple interest, Celsius-Fahrenheit conversion.

Precautions: Convert input, use parentheses, include units and test negative/decimal data where meaningful.

Lab 4: Selection programs

Odd/even, leap year, validated simple interest, day-name selection, menu-driven arithmetic.

Troubleshooting: Wrong branch usually means incorrect condition order or indentation.

Lab 5: Loop programs

Multiplication table and pattern printing.

Observation table: input, expected iterations, actual output, pass/fail.

Lab 6: Strings and collections

String split/count/case/reverse; list insert/append/delete/sort/slice; dictionary CRUD; set union/intersection/difference.

Inference: Select structure according to order, mutability and uniqueness requirements.

Lab 7: Functions and modules

Built-in functions, parameter passing, positional/default arguments and custom module creation.

Viva: Why import a function? To reuse tested code without copying it.

Standard practical record

Aim → CO → theory → algorithm/flowchart → code → test data → output → result → errors corrected → viva.

> Never record fabricated output. Run the program and capture the actual result.

EXPECTED / PRACTICE QUESTIONS

Module-wise Questions with Answers

Module I

Q: Distinguish algorithmic and heuristic methods.

Algorithmic methods follow definite steps and guarantee a result for valid input; heuristics are practical strategies that may be faster but do not guarantee an optimal result.

Q: Why convert `input()`?

It returns text. Numeric arithmetic requires `int()` or `float()`.

Module II

Q: Write logic for leap year.

```
year % 400 == 0 or (year % 4 == 0 and year % 100 != 0).
```

Q: Compare break and continue.

`break` ends the loop; `continue` skips only the current iteration.

Module III

Q: Why is a set useful for duplicate removal?

A set stores each hashable value only once.

Q: List versus tuple?

Both are ordered; a list is mutable while a tuple is immutable.

Module IV

Q: What is scope?

The region where a name can be accessed. Function variables are usually local; module-level names are global.

Q: Module versus package?

A module is one Python file; a package groups related modules.

Debugging question

Program prints repeated wrong output inside a loop. What should be checked?

Indentation, update statement, loop bounds, variable reset position and whether output belongs inside or after the loop.

Application question

Choose a structure for storing component code and rating.

A dictionary, e.g. `{"R101": "10 kΩ", "C201": "100 µF"}`, because each value is accessed by a meaningful key.

ASSESSMENT PRACTICE

Practice Model Paper — not an official SITTTR model question paper

1. Draw a flowchart to validate a number and classify it as positive, negative or zero.
2. Write a menu-driven arithmetic program using `match-case`.
3. Develop a program to count vowels and reverse a string.
4. Create a list program demonstrating insert, append, delete, sort and slice.
5. Create and import a custom module containing two calculation functions.

The supplied PDF gives assessment totals but does not provide an official question-paper structure or marks distribution.

QUICK REVISION

Exam and Practical Checklist

Before coding

Define input/output, constraints, algorithm and test data.

Before submission

Check indentation, types, boundary cases, labels, output units and comments.

Common failures

Unconverted input, wrong condition order, off-by-one range, infinite while loop, mutating while iterating.

Glossary

Course 1131

Term	Meaning
Algorithm	Finite ordered steps for solving a problem.
Flowchart	Graphical representation of control flow.
Interpreter	Software that executes Python code.
Iteration	Repeated execution of statements.
Mutable	Object can change after creation.
Function	Reusable named block of code.
Module	Python file containing definitions and statements.
Package	Organisation of related Python modules.

References

Official references: Not specified in the supplied official syllabus PDF.

Supplementary: Python 3 documentation available with the installed environment; instructor-approved IDE help and laboratory manual.

Search this handbook